

Простейшие примеры : Многофайловые проекты и пространства имен в C++

Пример 1: Самый простой многофайловый проект

Шаг 1: Создаем структуру проекта

```
простой_проект/  
├─ main.cpp      # Главный файл программы  
├─ math.cpp      # Реализация математических функций  
├─ math.h        # Заголовочный файл с объявлениями  
└─ greetings.cpp # Функции для приветствий
```

Шаг 2: Заголовочный файл (math.h)

```
// math.h - ЗАГОЛОВОЧНЫЙ ФАЙЛ  
// Заголовочные файлы содержат ОБЪЯВЛЕНИЯ (declarations)  
  
// Страж включения (include guard) - защита от двойного включения  
#ifndef MATH_H // Если MATH_H не определено...  
#define MATH_H // ...определяем MATH_H  
  
// Объявляем функции (говорим компилятору, что они существуют)  
int add(int a, int b); // Сложение  
int subtract(int a, int b); // Вычитание  
int multiply(int a, int b); // Умножение  
double divide(int a, int b); // Деление  
  
// КОНЕЦ стражей включения
```

```
#endif // MATH_H
```

```
/*
```

Что здесь происходит:

1. `#ifndef/#define/#endif` - защищают от повторного включения (
 2. Мы НЕ пишем реализацию функций здесь
 3. Мы только говорим: "Эти функции где-то есть, их можно испо
- ```
*/
```

### Шаг 3: Реализация математических функций (math.cpp)

```
// math.cpp - ИСХОДНЫЙ ФАЙЛ
```

```
// Исходные файлы содержат ОПРЕДЕЛЕНИЯ (definitions)
```

```
#include "math.h" // Включаем наш заголовочный файл
```

```
// Кавычки "" означают искать в текущей па
```

```
// Реализуем функцию сложения
```

```
int add(int a, int b) {
```

```
 return a + b; // Просто возвращаем сумму
```

```
}
```

```
// Реализуем функцию вычитания
```

```
int subtract(int a, int b) {
```

```
 return a - b; // Возвращаем разность
```

```
}
```

```
// Реализуем функцию умножения
```

```
int multiply(int a, int b) {
```

```
 return a * b; // Возвращаем произведение
```

```
}
```

```
// Реализуем функцию деления
```

```
double divide(int a, int b) {
```

```
 if (b == 0) { // Проверяем деление на ноль
```

```
 return 0; // Возвращаем 0 в случае ошибки
```

```
 }
```

```
 return static_cast<double>(a) / b; // Приводим к double ,
```

```
}
```

```
/*
```

Важные моменты:

1. Включаем `math.h`, чтобы компилятор знал объявления функций
2. Пишем ТОЧНО такие же сигнатуры функций, как в `math.h`
3. Добавляем реализацию (тело функции)

\*/

## Шаг 4: Еще один файл с функциями (`greetings.cpp`)

```
// greetings.cpp - Еще один исходный файл

#include <iostream> // Для использования cout
using namespace std; // Чтобы писать cout вместо std::cout

// Функция для приветствия
void sayHello() {
 cout << "Привет из greetings.cpp!" << endl;
}

// Функция для прощания
void sayGoodbye() {
 cout << "Пока! Заходи еще!" << endl;
}

// Функция, которая показывает использование математических ф
void showMathExample() {
 cout << "Пример из greetings.cpp:" << endl;
 // Здесь мы НЕ можем вызвать add() напрямую
 // потому что не подключили math.h
 // Но main.cpp подключит и math.h, и greetings.cpp
}
```

## Шаг 5: Главный файл программы (`main.cpp`)

```
// main.cpp - ГЛАВНЫЙ ФАЙЛ ПРОГРАММЫ
// Точка входа в программу

#include <iostream> // Для ввода/вывода
#include "math.h" // Наши математические функции
// greetings.cpp подключим при компиляции
```

```

using namespace std; // Чтобы не писать std:: перед cout

int main() { // Начало программы
 cout << "=== ПРОСТОЙ МНОГОФАЙЛОВЫЙ ПРОЕКТ ===" << endl;
 cout << "===== " << endl;

 // Используем математические функции
 cout << "\nМатематические операции:" << endl;
 cout << "5 + 3 = " << add(5, 3) << endl;
 cout << "10 - 4 = " << subtract(10, 4) << endl;
 cout << "6 * 7 = " << multiply(6, 7) << endl;
 cout << "15 / 4 = " << divide(15, 4) << endl;

 // Проверка деления на ноль
 cout << "8 / 0 = " << divide(8, 0) << " (защита от делени:

 // Здесь мы не можем вызвать sayHello() напрямую,
 // потому что не объявили ее в greetings.h

 cout << "\nПрограмма завершена успешно!" << endl;

 return 0; // Возвращаем 0 - все хорошо
}

```

## Шаг 6: Компиляция и запуск

```

Откройте терминал в папке проекта

Компилируем все файлы вместе:
g++ main.cpp math.cpp greetings.cpp -o my_program

Запускаем программу:
./my_program

Или пошагово (как делает IDE):
1. Компилируем каждый .cpp файл в .o файл:
g++ -c main.cpp # создает main.o
g++ -c math.cpp # создает math.o
g++ -c greetings.cpp # создает greetings.o

```

```
2. Связываем все .o файлы вместе:
g++ main.o math.o greetings.o -o my_program

3. Запускаем:
./my_program
```

## Пример 2: Добавляем заголовочный файл для greetings

### Шаг 1: Создаем greetings.h

```
// greetings.h - Заголовочный файл для приветствий

#ifndef GREETINGS_H
#define GREETINGS_H

// Объявляем функции
void sayHello();
void sayGoodbye();
void showMathExample();

#endif // GREETINGS_H
```

### Шаг 2: Обновляем greetings.cpp

```
// greetings.cpp - теперь с заголовочным файлом

#include <iostream>
#include "greetings.h" // Включаем наш заголовочный файл

using namespace std;

void sayHello() {
 cout << "Привет! Добро пожаловать в нашу программу!" << endl;
}

void sayGoodbye() {
```

```

 cout << "Спасибо за использование! До свидания!" << endl;
 }

 void showMathExample() {
 cout << "Здесь могла бы быть математика..." << endl;
 }

```

### Шаг 3: Обновляем main.cpp

```

// main.cpp - теперь с greetings.h

#include <iostream>
#include "math.h"
#include "greetings.h" // Добавили заголовочный файл

using namespace std;

int main() {
 cout << "=== УЛУЧШЕННЫЙ МНОГОФАЙЛОВЫЙ ПРОЕКТ ===" << endl

 // Используем функции из greetings.cpp
 sayHello();

 cout << "\nМатематика:" << endl;
 int x = 20, y = 5;
 cout << x << " + " << y << " = " << add(x, y) << endl;
 cout << x << " * " << y << " = " << multiply(x, y) << endl

 showMathExample();
 sayGoodbye();

 return 0;
}

```

### Шаг 4: Компилируем и запускаем

```

Теперь у нас 4 файла для компиляции
g++ main.cpp math.cpp greetings.cpp -o program_v2

```

./program\_v2

## Пример 3: Вводим пространства имен (namespace)

### Шаг 1: Создаем проект с пространствами имен

```
проект_c_namespace/
├─ main.cpp
├─ calculator.cpp
├─ calculator.h
├─ greetings.cpp
└─ greetings.h
```

### Шаг 2: calculator.h с namespace

```
// calculator.h - Калькулятор с пространством имен

#ifndef CALCULATOR_H
#define CALCULATOR_H

// Создаем пространство имен Calculator
namespace Calculator {
 // Внутри namespace объявляем функции
 int add(int a, int b);
 int subtract(int a, int b);
 int multiply(int a, int b);
 double divide(int a, int b);

 // Можно объявить константу
 const double PI = 3.14159;

 // Можно объявить вложенный namespace
 namespace Advanced {
 int power(int base, int exponent);
 double squareRoot(double number);
 }
}
```

```
}

#endif // CALCULATOR_H
```

### Шаг 3: calculator.cpp с реализацией

```
// calculator.cpp - Реализация Calculator

#include "calculator.h"
#include <cmath> // Для sqrt() и pow()

// Реализуем функции из namespace Calculator
namespace Calculator {
 int add(int a, int b) {
 return a + b;
 }

 int subtract(int a, int b) {
 return a - b;
 }

 int multiply(int a, int b) {
 return a * b;
 }

 double divide(int a, int b) {
 if (b == 0) return 0;
 return static_cast<double>(a) / b;
 }

 // Реализуем функции вложенного namespace
 namespace Advanced {
 int power(int base, int exponent) {
 int result = 1;
 for (int i = 0; i < exponent; i++) {
 result *= base;
 }
 return result;
 }

 double squareRoot(double number) {
```



```
 if (number < 0) return 0;
 return sqrt(number);
 }
}
```

## Шаг 4: greetings.h с namespace

```
// greetings.h - Приветствия с namespace

#ifndef GREETINGS_H
#define GREETINGS_H

#include <string> // Для std::string

// Пространство имен для работы с приветствиями
namespace Greetings {
 // Функция, которая возвращает приветствие
 std::string getHelloMessage();

 // Функция с параметром
 std::string getPersonalHello(const std::string& name);

 // Функция для прощания
 std::string getGoodbyeMessage();

 // Внутренняя функция (не экспортируем в .h)
 // static или в отдельном файле реализации
}

#endif // GREETINGS_H
```

## Шаг 5: greetings.cpp с реализацией

```
// greetings.cpp - Реализация Greetings

#include "greetings.h"
#include <iostream>
#include <string>
```

```
using namespace std;

namespace Greetings {
 // Простая функция
 string getHelloMessage() {
 return "Здравствуйтесь!";
 }

 // Функция с параметром
 string getPersonalHello(const string& name) {
 return "Привет, " + name + "! Рад тебя видеть!";
 }

 // Еще одна функция
 string getGoodbyeMessage() {
 return "Всего доброго! Приходи еще!";
 }

 // Внутренняя вспомогательная функция
 // (не объявлена в .h, поэтому видна только здесь)
 string toUpperCase(const string& text) {
 string result = text;
 for (char& c : result) {
 if (c >= 'a' && c <= 'z') {
 c = c - 'a' + 'A';
 }
 }
 return result;
 }
}
```

## Шаг 6: main.cpp с использованием namespace

```
// main.cpp - Главный файл с использованием namespace

#include <iostream>
#include <string>
#include "calculator.h"
#include "greetings.h"
```

// Разные способы использования namespace

```
int main() {
 std::cout << "=== ПРОЕКТ С ПРОСТРАНСТВАМИ ИМЕН ===" << std::endl;
 std::cout << "===== " << std::endl;

 // СПОСОБ 1: Полное имя с ::
 std::cout << "\n1. Использование с полным именем:" << std::endl;
 std::cout << "5 + 3 = " << Calculator::add(5, 3) << std::endl;
 std::cout << "Приветствие: " << Greetings::getHelloMessage() << std::endl;

 // СПОСОБ 2: Using declaration (для конкретных функций)
 using Calculator::multiply;
 using Greetings::getPersonalHello;

 std::cout << "\n2. Using declaration:" << std::endl;
 std::cout << "6 * 7 = " << multiply(6, 7) << std::endl;
 std::cout << getPersonalHello("Анна") << std::endl;

 // СПОСОБ 3: Using directive (для всего namespace)
 {
 using namespace Calculator;
 std::cout << "\n3. Using directive (в блоке):" << std::endl;
 std::cout << "10 - 4 = " << subtract(10, 4) << std::endl;
 std::cout << "PI = " << PI << std::endl;
 }

 // Вложенные namespace
 std::cout << "\n4. Вложенные namespace:" << std::endl;
 std::cout << "2^3 = " << Calculator::Advanced::power(2, 3) << std::endl;
 std::cout << "√16 = " << Calculator::Advanced::squareRoot(16) << std::endl;

 // Конфликт имен (если бы были)
 int add = 100; // Локальная переменная
 std::cout << "\n5. Конфликт имен:" << std::endl;
 std::cout << "Локальная переменная add = " << add << std::endl;
 std::cout << "Функция Calculator::add(2, 3) = " << Calculator::add(2, 3) << std::endl;

 // Псевдонимы для длинных имен
 namespace Calc = Calculator;
 namespace CalcAdv = Calculator::Advanced;

 std::cout << "\n6. Псевдонимы (aliases):" << std::endl;
 std::cout << Calc::add(2, 3) << std::endl;
 std::cout << CalcAdv::power(2, 3) << std::endl;
 std::cout << CalcAdv::squareRoot(16) << std::endl;
}
```

```

std::cout << "Calc::add(10, 20) = " << Calc::add(10, 20) << "\n";
std::cout << "CalcAdv::power(3, 2) = " << CalcAdv::power(3, 2) << "\n";

std::cout << "\n" << Greetings::getGoodbyeMessage() << "\n";

return 0;
}

```

## Шаг 7: Компиляция и запуск

```

Компилируем все вместе
g++ main.cpp calculator.cpp greetings.cpp -o namespace_project

Запускаем
./namespace_project

```



## Пример 4: Практический проект - Учет студентов

### Структура проекта:

```

студенты_проект/
├── main.cpp
├── student.cpp
├── student.h
├── database.cpp
└── database.h

```

### Шаг 1: student.h

```

// student.h - Класс Студент

#ifndef STUDENT_H
#define STUDENT_H

#include <string>

```

```
namespace StudentSystem {

 class Student {
 private:
 std::string name;
 int age;
 double gpa;

 public:
 // Конструктор
 Student(std::string studentName, int studentAge, double studentGpa) : name(studentName), age(studentAge), gpa(studentGpa) {}

 // Методы получения данных
 std::string getName() const;
 int getAge() const;
 double getGPA() const;

 // Методы изменения данных
 void setName(const std::string& newName);
 void setAge(int newAge);
 void setGPA(double newGpa);

 // Метод для вывода информации
 void displayInfo() const;

 // Проверка, является ли студент отличником
 bool isExcellent() const;
 };

} // namespace StudentSystem

#endif // STUDENT_H
```

## Шар 2: student.cpp

```
// student.cpp - Реализация класса Student

#include "student.h"
#include <iostream>
```

```
using namespace std;

namespace StudentSystem {

 // Конструктор
 Student::Student(string studentName, int studentAge, double studentGpa)
 : name(studentName), age(studentAge), gpa(studentGpa)
 {}

 // Геттеры (получение данных)
 string Student::getName() const {
 return name;
 }

 int Student::getAge() const {
 return age;
 }

 double Student::getGPA() const {
 return gpa;
 }

 // Сеттеры (изменение данных)
 void Student::setName(const string& newName) {
 name = newName;
 }

 void Student::setAge(int newAge) {
 if (newAge > 0 && newAge < 120) {
 age = newAge;
 }
 }

 void Student::setGPA(double newGpa) {
 if (newGpa >= 0.0 && newGpa <= 4.0) {
 gpa = newGpa;
 }
 }

 // Вывод информации
 void Student::displayInfo() const {
 cout << "Студент: " << name << endl;
 cout << "Возраст: " << age << " лет" << endl;
 }
}
```

```

 cout << "Средний балл: " << gpa << endl;

 if (isExcellent()) {
 cout << "Статус: ОТЛИЧНИК" << endl;
 }
 }

 // Проверка на отличника
 bool Student::isExcellent() const {
 return gpa >= 3.5;
 }

} // namespace StudentSystem

```

### Шар 3: database.h

```

// database.h - База данных студентов

#ifndef DATABASE_H
#define DATABASE_H

#include "student.h"
#include <vector>

namespace StudentSystem {

 class StudentDatabase {
 private:
 std::vector<Student> students;

 public:
 // Добавление студента
 void addStudent(const Student& student);

 // Удаление студента по имени
 bool removeStudent(const std::string& name);

 // Поиск студента по имени
 Student* findStudent(const std::string& name);

 // Получение всех студентов

```

```

 const std::vector<Student>& getAllStudents() const;

 // Получение отличников
 std::vector<Student> getExcellentStudents() const;

 // Подсчет студентов
 int getStudentCount() const;

 // Вывод всех студентов
 void displayAllStudents() const;
};

} // namespace StudentSystem

#endif // DATABASE_H

```

#### Шаг 4: database.cpp

```

// database.cpp - Реализация базы данных

#include "database.h"
#include <iostream>

using namespace std;

namespace StudentSystem {

 void StudentDatabase::addStudent(const Student& student) {
 students.push_back(student);
 cout << "Студент " << student.getName() << " добавлен\n";
 }

 bool StudentDatabase::removeStudent(const string& name) {
 for (auto it = students.begin(); it != students.end(); ++it) {
 if (*it).getName() == name) {
 students.erase(it);
 cout << "Студент " << name << " удален из базы\n";
 return true;
 }
 }
 cout << "Студент " << name << " не найден.\n";
 }
}

```



```

 return false;
 }

Student* StudentDatabase::findStudent(const string& name)
 for (auto& student : students) {
 if (student.getName() == name) {
 return &student;
 }
 }
 return nullptr; // nullptr означает "ничего не найде
}

const vector<Student>& StudentDatabase::getAllStudents() {
 return students;
}

vector<Student> StudentDatabase::getExcellentStudents() const {
 vector<Student> excellent;
 for (const auto& student : students) {
 if (student.isExcellent()) {
 excellent.push_back(student);
 }
 }
 return excellent;
}

int StudentDatabase::getStudentCount() const {
 return students.size();
}

void StudentDatabase::displayAllStudents() const {
 if (students.empty()) {
 cout << "В базе данных нет студентов." << endl;
 return;
 }

 cout << "\n=== ВСЕ СТУДЕНТЫ ===" << endl;
 cout << "Всего: " << students.size() << " студентов" << endl;
 cout << "===== " << endl;

 for (const auto& student : students) {
 student.displayInfo();
 cout << "---" << endl;
 }
}

```

```
 }
}

} // namespace StudentSystem
```

## Шаг 5: main.cpp

```
// main.cpp - Главная программа

#include <iostream>
#include "student.h"
#include "database.h"

using namespace std;
using namespace StudentSystem; // Используем наше пространство имен

int main() {
 cout << "=== СИСТЕМА УЧЕТА СТУДЕНТОВ ===" << endl;
 cout << "===== " << endl;

 // Создаем базу данных
 StudentDatabase database;

 // Создаем студентов
 Student student1("Иван Иванов", 20, 3.8);
 Student student2("Анна Петрова", 21, 4.0);
 Student student3("Петр Сидоров", 19, 2.9);
 Student student4("Мария Кузнецова", 22, 3.6);

 // Добавляем студентов в базу
 database.addStudent(student1);
 database.addStudent(student2);
 database.addStudent(student3);
 database.addStudent(student4);

 // Показываем всех студентов
 database.displayAllStudents();

 // Ищем конкретного студента
 cout << "\n=== ПОИСК СТУДЕНТА ===" << endl;
 Student* found = database.findStudent("Анна Петрова");
```

```
if (found != nullptr) {
 cout << "Найден студент:" << endl;
 found->displayInfo();
} else {
 cout << "Студент не найден." << endl;
}

// Показываем только отличников
cout << "\n=== ОТЛИЧНИКИ ===" << endl;
vector<Student> excellent = database.getExcellentStudents
cout << "Отличников: " << excellent.size() << endl;
for (const auto& student : excellent) {
 student.displayInfo();
 cout << endl;
}

// Удаляем студента
cout << "\n=== УДАЛЕНИЕ СТУДЕНТА ===" << endl;
database.removeStudent("Петр Сидоров");

// Показываем обновленный список
cout << "\n=== ОБНОВЛЕННЫЙ СПИСОК ===" << endl;
database.displayAllStudents();

// Изменяем данные студента
cout << "\n=== ИЗМЕНЕНИЕ ДАННЫХ ===" << endl;
Student* ivan = database.findStudent("Иван Иванов");
if (ivan != nullptr) {
 cout << "До изменения:" << endl;
 ivan->displayInfo();

 ivan->setGPA(4.0); // Повысили успеваемость!

 cout << "\nПосле изменения:" << endl;
 ivan->displayInfo();
}

cout << "\nПрограмма завершена." << endl;
return 0;
}
```

## Шаг 6: Компиляция и запуск

```
Компилируем
g++ main.cpp student.cpp database.cpp -o student_system

Запускаем
./student_system
```

## Пример 5: Советы и лучшие практики

### Файл: best\_practices.cpp

```
// best_practices.cpp - Лучшие практики для новичков

#include <iostream>
#include <string>

/*
СОВЕТ 1: Организация проекта

ХОРОШО: ПЛОХО:
project/ project.cpp (все в одном файле)
├─ main.cpp
├─ module1.cpp
├─ module1.h
├─ module2.cpp
└─ module2.h
*/

/*
СОВЕТ 2: Заголовочные файлы (.h)

В .h файлах:
1. Только объявления (declarations)
2. Никогда не пишите реализацию (кроме шаблонов)
3. Всегда используйте #ifndef/#define/#endif
4. Минимизируйте #include в .h файлах
*/

/*
СОВЕТ 3: Пространства имен
```

-----

Используйте namespace для:

1. Группировки связанного кода
2. Избегания конфликтов имен
3. Создания логической структуры

Пример хорошей структуры:

```
namespace CompanyName {
 namespace ProjectName {
 namespace ModuleName {
 // ваш код
 }
 }
}
*/
```

/\*

СОВЕТ 4: Именование файлов

-----

ХОРОШИЕ ИМЕНА:

- calculator.h
- student\_database.cpp
- math\_utils.cpp
- config\_settings.h

\*/

ПЛОХИЕ ИМЕНА:

- file1.h
- mycode.cpp
- project.cpp
- header.h

// Простой пример хорошей структуры

```
namespace GoodExample {

 // math_operations.h будет содержать:
 namespace Math {
 int add(int a, int b);
 int multiply(int a, int b);
 }

 // string_utils.h будет содержать:
 namespace StringUtils {
 std::string toUpperCase(const std::string& str);
 std::string trim(const std::string& str);
 }

}
```

```
int main() {
 std::cout << "=== ЛУЧШИЕ ПРАКТИКИ ===" << std::endl;
 std::cout << "===== " << std::endl;

 std::cout << "\n1. Разделяйте код на логические модули" << std::endl;
 std::cout << "2. Используйте понятные имена файлов" << std::endl;
 std::cout << "3. Всегда защищайте заголовочные файлы" << std::endl;
 std::cout << "4. Используйте пространства имен" << std::endl;
 std::cout << "5. Комментируйте ваш код" << std::endl;

 return 0;
}
```

## Краткое руководство для новичков:

### Как создать многофайловый проект:

#### 1. Разделите код на логические части

- Каждый модуль = пара .h и .cpp файлов
- Главная программа = main.cpp

#### 2. Создайте заголовочный файл (.h):

```
#ifndef ИМЯ_ФАЙЛА_H
#define ИМЯ_ФАЙЛА_H

// Объявления функций/классов

#endif
```

#### 3. Создайте исходный файл (.cpp):

```
#include "имя_файла.h"

// Реализации функций/классов
```

#### 4. Скомпилируйте все вместе:

```
g++ main.cpp файл1.cpp файл2.cpp -o программа
```

## Как использовать пространства имен:

### 1. Объявите в .h файле:

```
namespace MyNamespace {
 void myFunction();
 class MyClass {};
}
```

### 2. Реализуйте в .cpp файле:

```
namespace MyNamespace {
 void myFunction() {
 // реализация
 }
}
```

### 3. Используйте в main.cpp:

```
// Способ 1: Полное имя
MyNamespace::myFunction();

// Способ 2: Using declaration
using MyNamespace::myFunction;
myFunction();

// Способ 3: Using directive (осторожно!)
using namespace MyNamespace;
myFunction();
```

## Примеры: Многофайловые проекты и пространства имен

# Пример 1: Самый простой проект с массивами

## Структура проекта:

```
проект1/
├─ main.cpp # Главная программа
├─ array_tools.cpp # Функции для работы с массивами
└─ array_tools.h # Объявления функций
```

## Шаг 1: array\_tools.h - заголовочный файл

```
// array_tools.h - ЗАГОЛОВОЧНЫЙ файл
// Этот файл говорит: "Какие функции у нас есть"

// Защита от двойного включения (если файл уже включен, пропус
#ifndef ARRAY_TOOLS_H // Если ARRAY_TOOLS_H не определен
#define ARRAY_TOOLS_H // То определить ARRAY_TOOLS_H

// Объявляем функции (говорим, что они существуют)

// Функция печати массива
void printArray(int arr[], int size); // arr[] - массив, size

// Функция заполнения массива случайными числами
void fillRandom(int arr[], int size, int min, int max);
// min - минимальное число, max - максимальное

// Функция нахождения суммы элементов массива
int sumArray(int arr[], int size); // Возвращает сумму (int)

// Функция нахождения максимального элемента
int findMax(int arr[], int size); // Возвращает максимальное

// Конец защиты
#endif // ARRAY_TOOLS_H
```

## Шаг 2: array\_tools.cpp - реализация функций



```
// array_tools.cpp - ИСХОДНЫЙ файл
// Здесь пишем РЕАЛИЗАЦИЮ функций (их код)

#include "array_tools.h" // Включаем наш заголовочный файл
#include <iostream> // Для cout (вывода на экран)
#include <cstdlib> // Для rand() (случайные числа)
#include <ctime> // Для time() (чтобы rand() работал

using namespace std; // Чтобы писать cout вместо std::cout

// Реализация функции печати массива
void printArray(int arr[], int size) {
 cout << "Массив: [";
 for (int i = 0; i < size; i++) { // Проходим по всем эле
 cout << arr[i]; // Выводим элемент
 if (i < size - 1) { // Если не последний эле
 cout << ", "; // Ставим запятую
 }
 }
 cout << "]" << endl; // Закрываем скобку и п
}

// Реализация функции заполнения случайными числами
void fillRandom(int arr[], int size, int min, int max) {
 srand(time(0)); // Инициализация генератора случайных чис
 for (int i = 0; i < size; i++) {
 // Генерируем число от min до max
 arr[i] = min + rand() % (max - min + 1);
 }
}

// Реализация функции суммы
int sumArray(int arr[], int size) {
 int sum = 0; // Переменная для суммы
 for (int i = 0; i < size; i++) {
 sum += arr[i]; // Добавляем каждый элемент к сумме
 }
 return sum; // Возвращаем результат
}

// Реализация функции поиска максимума
int findMax(int arr[], int size) {
```

```

 if (size == 0) return 0; // Если массив пустой, возвращае

 int max = arr[0]; // Предполагаем, что первый элемент -
 for (int i = 1; i < size; i++) { // Начинаем со второго :
 if (arr[i] > max) { // Если нашли элемент больше
 max = arr[i]; // Запоминаем его
 }
 }
 return max; // Возвращаем максимум
}

```

### Шаг 3: main.cpp - главная программа

```

// main.cpp - ГЛАВНАЯ программа
// Здесь мы ИСПОЛЬЗУЕМ наши функции

#include <iostream> // Для ввода/вывода
#include "array_tools.h" // Включаем наши функции для работы с массивами

using namespace std; // Чтобы не писать std:: перед cout

int main() { // Начало программы
 cout << "=== ПРОСТОЙ ПРОЕКТ С МАССИВАМИ ===" << endl;
 cout << "===== " << endl;

 // 1. Создаем массив из 5 чисел
 const int SIZE = 5; // Константа - размер массива (нельзя изменить)
 int numbers[SIZE]; // Объявляем массив из 5 элементов

 // 2. Заполняем массив случайными числами от 1 до 10
 fillRandom(numbers, SIZE, 1, 10);

 // 3. Печатаем массив
 printArray(numbers, SIZE);

 // 4. Находим и выводим сумму элементов
 int total = sumArray(numbers, SIZE);
 cout << "Сумма элементов: " << total << endl;

 // 5. Находим и выводим максимальный элемент
 int maximum = findMax(numbers, SIZE);
}

```

```

 cout << "Максимальный элемент: " << maximum << endl;

 // 6. Создаем еще один массив
 cout << "\n=== ВТОРОЙ МАССИВ ===" << endl;
 int anotherArray[3] = {10, 20, 30}; // Создаем и сразу записываем значения

 // 7. Используем наши функции с новым массивом
 printArray(anotherArray, 3);
 cout << "Сумма: " << sumArray(anotherArray, 3) << endl;
 cout << "Максимум: " << findMax(anotherArray, 3) << endl;

 cout << "\nПрограмма завершена!" << endl;
 return 0; // Конец программы (все хорошо)
}

```

## Шаг 4: Компиляция и запуск

```

Откройте терминал в папке проекта

Компилируем оба .cpp файла вместе:
g++ main.cpp array_tools.cpp -o array_project

Запускаем программу:
./array_project

Вывод будет примерно таким:
=== ПРОСТОЙ ПРОЕКТ С МАССИВАМИ ===
=====
Массив: [3, 7, 2, 9, 5]
Сумма элементов: 26
Максимальный элемент: 9
#
=== ВТОРОЙ МАССИВ ===
Массив: [10, 20, 30]
Сумма: 60
Максимум: 30
#
Программа завершена!

```



## Пример 2: Проект с пространствами имен

### Структура проекта:

```
проект2/
├─ main.cpp
├─ math_array.cpp
├─ math_array.h
├─ string_array.cpp
└─ string_array.h
```

### Шаг 1: math\_array.h - математические операции с массивами

```
// math_array.h - Математические функции для массивов

#ifndef MATH_ARRAY_H
#define MATH_ARRAY_H

// Создаем ПРОСТРАНСТВО ИМЕН MathArray
// Все функции внутри будут принадлежать этому пространству
namespace MathArray {
 // Функция сложения двух массивов
 void addArrays(int arr1[], int arr2[], int result[], int size);
 // arr1 - первый массив, arr2 - второй
 // result - массив для результата, size - размер всех массивов

 // Функция умножения массива на число
 void multiplyByNumber(int arr[], int size, int number, int result[]);

 // Функция нахождения среднего значения
 double findAverage(int arr[], int size); // Возвращает double

 // Функция проверки, отсортирован ли массив
 bool isSorted(int arr[], int size); // Возвращает true или false
}

#endif
```



## Шаг 2: math\_array.cpp - реализация

```
// math_array.cpp - Реализация математических функций

#include "math_array.h" // Наш заголовочный файл
#include <iostream>

using namespace std;

// Указываем, что реализуем функции из пространства имен Math
namespace MathArray {

 void addArrays(int arr1[], int arr2[], int result[], int size) {
 for (int i = 0; i < size; i++) {
 result[i] = arr1[i] + arr2[i]; // Складываем соответствующие элементы
 }
 }

 void multiplyByNumber(int arr[], int size, int number, int result[]) {
 for (int i = 0; i < size; i++) {
 result[i] = arr[i] * number; // Умножаем каждый элемент на число
 }
 }

 double findAverage(int arr[], int size) {
 if (size == 0) return 0.0; // Защита от деления на 0

 int sum = 0;
 for (int i = 0; i < size; i++) {
 sum += arr[i];
 }
 return static_cast<double>(sum) / size; // Приводим к типу double
 }

 bool isSorted(int arr[], int size) {
 if (size < 2) return true; // Массив из 0 или 1 элемента считается отсортированным

 // Проверяем, отсортирован ли по возрастанию
 for (int i = 1; i < size; i++) {
 if (arr[i] < arr[i - 1]) { // Если текущий меньше предыдущего
 return false; // Не отсортирован
 }
 }
 }
}
```

```

 }
 return true; // Если дошли до конца, значит отсортир
}
}

```

### Шаг 3: string\_array.h - работа с массивами строк

```

// string_array.h - Функции для массивов строк

#ifndef STRING_ARRAY_H
#define STRING_ARRAY_H

#include <string> // Для использования std::string

// Другое пространство имен
namespace StringArray {
 // Функция печати массива строк
 void printStringArray(std::string arr[], int size);

 // Функция поиска самой длинной строки
 std::string findLongestString(std::string arr[], int size);

 // Функция подсчета строк, начинающихся с буквы
 int countStringsStartingWith(std::string arr[], int size,
 // letter - буква, с которой должна начинаться строка

 // Функция объединения всех строк в одну
 std::string joinStrings(std::string arr[], int size, std:
 // separator - разделитель между строками
}

#endif

```

### Шаг 4: string\_array.cpp - реализация

```

// string_array.cpp - Реализация функций для строк

#include "string_array.h"

```

```
#include <iostream>

using namespace std;

namespace StringArray {

 void printStringArray(string arr[], int size) {
 cout << "Строки: ";
 for (int i = 0; i < size; i++) {
 cout << "\"" << arr[i] << "\""; // Выводим в кавычки
 if (i < size - 1) cout << ", ";
 }
 cout << endl;
 }

 string findLongestString(string arr[], int size) {
 if (size == 0) return ""; // Если массив пустой

 string longest = arr[0]; // Первая строка - пока самая длинная
 for (int i = 1; i < size; i++) {
 if (arr[i].length() > longest.length()) { // Сравнение длин
 longest = arr[i]; // Нашли более длинную
 }
 }
 return longest;
 }

 int countStringsStartingWith(string arr[], int size, char letter) {
 int count = 0; // Счетчик
 for (int i = 0; i < size; i++) {
 if (!arr[i].empty() && arr[i][0] == letter) { // Проверка на пустоту и совпадение
 count++; // Увеличиваем счетчик
 }
 }
 return count;
 }

 string joinStrings(string arr[], int size, string separator) {
 if (size == 0) return "";

 string result = arr[0]; // Начинаем с первой строки
 for (int i = 1; i < size; i++) {
 result += separator + arr[i]; // Добавляем разделитель
 }
 }
}
```

```

 }
 return result;
}
}

```

## Шаг 5: main.cpp - использование всех функций

```

// main.cpp - Главная программа с использованием пространств

#include <iostream>
#include "math_array.h"
#include "string_array.h"

using namespace std;

int main() {
 cout << "=== ПРОЕКТ С ПРОСТРАНСТВАМИ ИМЕН ===" << endl;
 cout << "===== " << endl;

 // ЧАСТЬ 1: Математические операции с массивами чисел
 cout << "\n1. МАТЕМАТИЧЕСКИЕ ОПЕРАЦИИ:" << endl;
 cout << "-----" << endl;

 int numbers1[5] = {1, 2, 3, 4, 5};
 int numbers2[5] = {10, 20, 30, 40, 50};
 int result[5];

 // Использование функций из MathArray
 MathArray::addArrays(numbers1, numbers2, result, 5);
 cout << "Сложение массивов: ";
 for (int i = 0; i < 5; i++) {
 cout << result[i] << " ";
 }
 cout << endl;

 MathArray::multiplyByNumber(numbers1, 5, 2, result);
 cout << "Умножение на 2: ";
 for (int i = 0; i < 5; i++) {
 cout << result[i] << " ";
 }
 cout << endl;
}

```



```

double avg = MathArray::findAverage(numbers1, 5);
cout << "Среднее чисел 1-5: " << avg << endl;

bool sorted = MathArray::isSorted(numbers1, 5);
cout << "Массив отсортирован? " << (sorted ? "Да" : "Нет") << endl;

// ЧАСТЬ 2: Операции с массивами строк
cout << "\n2. ОПЕРАЦИИ СО СТРОКАМИ:" << endl;
cout << "-----" << endl;

string fruits[4] = {"яблоко", "банан", "апельсин", "арбуз"};

// Использование функций из StringArray
StringArray::printStringArray(fruits, 4);

string longest = StringArray::findLongestString(fruits, 4);
cout << "Самая длинная строка: " << longest << endl;

int countA = StringArray::countStringsStartingWith(fruits, 'a');
cout << "Строк на 'a': " << countA << endl;

string joined = StringArray::joinStrings(fruits, 4, ", ");
cout << "Объединенные строки: " << joined << endl;

// ЧАСТЬ 3: Разные способы использования namespace
cout << "\n3. РАЗНЫЕ СПОСОБЫ ИСПОЛЬЗОВАНИЯ:" << endl;
cout << "-----" << endl;

// Способ 1: Полное имя (уже использовали выше)
cout << "Способ 1: MathArray::findAverage = "
 << MathArray::findAverage(numbers2, 5) << endl;

// Способ 2: Using declaration (для конкретной функции)
using MathArray::isSorted;
int testArray[3] = {3, 1, 2};
cout << "Способ 2: isSorted? " << (isSorted(testArray, 3)) << endl;

// Способ 3: Using directive (для всех функций в namespace)
{
 using namespace StringArray;
 string colors[3] = {"красный", "зеленый", "синий"};
 cout << "Способ 3: Самая длинный цвет: " << findLongestString(colors, 3) << endl;
}

```

```

 }

 cout << "\nПрограмма завершена!" << endl;
 return 0;
}

```

## Шаг 6: Компиляция и запуск

```

Компилируем ВСЕ .cpp файлы:
g++ main.cpp math_array.cpp string_array.cpp -o namespace_pro

Запускаем:
./namespace_project

Вывод будет примерно таким:
=== ПРОЕКТ С ПРОСТРАНСТВАМИ ИМЕН ===
=====
#
1. МАТЕМАТИЧЕСКИЕ ОПЕРАЦИИ:

Сложение массивов: 11 22 33 44 55
Умножение на 2: 2 4 6 8 10
Среднее чисел 1-5: 3
Массив отсортирован? Да
#
2. ОПЕРАЦИИ СО СТРОКАМИ:

Строки: "яблоко", "банан", "апельсин", "арбуз"
Самая длинная строка: апельсин
Строк на 'а': 2
Объединенные строки: яблоко, банан, апельсин, арбуз
#
3. РАЗНЫЕ СПОСОБЫ ИСПОЛЬЗОВАНИЯ:

Способ 1: MathArray::findAverage = 30
Способ 2: isSorted? Нет
Способ 3: Самая длинный цвет: красный
#
Программа завершена!

```



## Пример 3: Еще проще - проект с оценками студентов

### Структура проекта:

```
проект3/
├─ main.cpp
├─ grades.cpp
└─ grades.h
```

### grades.h:

```
// grades.h - Работа с оценками студентов

#ifndef GRADES_H
#define GRADES_H

namespace Grades {
 // Функция вычисления средней оценки
 float calculateAverage(int grades[], int count);
 // grades - массив оценок, count - количество оценок

 // Функция нахождения самой высокой оценки
 int findHighest(int grades[], int count);

 // Функция нахождения самой низкой оценки
 int findLowest(int grades[], int count);

 // Функция проверки, есть ли неудовлетворительные оценки
 bool hasFailed(int grades[], int count, int passingGrade :
 // passingGrade - проходной балл (по умолчанию 3)

 // Функция подсчета отличников
 int countExcellent(int grades[], int count, int excellent)
}

#endif
```



## grades.cpp:

```
// grades.cpp - Реализация функций для оценок

#include "grades.h"

namespace Grades {

 float calculateAverage(int grades[], int count) {
 if (count == 0) return 0.0f;

 int sum = 0;
 for (int i = 0; i < count; i++) {
 sum += grades[i];
 }
 return static_cast<float>(sum) / count;
 }

 int findHighest(int grades[], int count) {
 if (count == 0) return 0;

 int highest = grades[0];
 for (int i = 1; i < count; i++) {
 if (grades[i] > highest) {
 highest = grades[i];
 }
 }
 return highest;
 }

 int findLowest(int grades[], int count) {
 if (count == 0) return 0;

 int lowest = grades[0];
 for (int i = 1; i < count; i++) {
 if (grades[i] < lowest) {
 lowest = grades[i];
 }
 }
 return lowest;
 }
}
```

```

bool hasFailed(int grades[], int count, int passingGrade)
{
 for (int i = 0; i < count; i++) {
 if (grades[i] < passingGrade) {
 return true; // Нашли неудовлетворительную оц
 }
 }
 return false; // Все оценки удовлетворительные
}

int countExcellent(int grades[], int count, int excellentGrade)
{
 int excellentCount = 0;
 for (int i = 0; i < count; i++) {
 if (grades[i] == excellentGrade) {
 excellentCount++;
 }
 }
 return excellentCount;
}
}

```

### main.cpp:

```

// main.cpp - Программа для анализа оценок

#include <iostream>
#include "grades.h"

using namespace std;

int main() {
 cout << "=== АНАЛИЗ ОЦЕНОК СТУДЕНТОВ ===" << endl;

 // Оценки группы студентов
 int studentGrades[10] = {5, 4, 3, 5, 2, 4, 3, 5, 4, 3};

 cout << "Оценки: ";
 for (int i = 0; i < 10; i++) {
 cout << studentGrades[i] << " ";
 }
 cout << endl;
}

```

```
// Используем функции из namespace Grades
float average = Grades::calculateAverage(studentGrades, 10);
cout << "Средняя оценка: " << average << endl;

int highest = Grades::findHighest(studentGrades, 10);
cout << "Самая высокая оценка: " << highest << endl;

int lowest = Grades::findLowest(studentGrades, 10);
cout << "Самая низкая оценка: " << lowest << endl;

bool failed = Grades::hasFailed(studentGrades, 10);
cout << "Есть неудовлетворительные? " << (failed ? "да" :

int excellent = Grades::countExcellent(studentGrades, 10);
cout << "Количество отличников (5): " << excellent << endl;

return 0;
}
```

## Компиляция:

```
g++ main.cpp grades.cpp -o grades_project
./grades_project
```

## Ключевые моменты для новичков:

### 1. Зачем нужны многофайловые проекты?

- **Упорядочивание кода** - каждый файл отвечает за свою часть
- **Повторное использование** - можно использовать функции в разных проектах
- **Упрощение работы** - проще искать и исправлять ошибки

### 2. Основные типы файлов:

- **.h (header)** - заголовочные файлы (объявления)
- **.cpp (source)** - исходные файлы (реализации)

- **main.cpp** - главная программа

### 3. Что такое пространства имен (namespace)?

- **Контейнер для функций** - группирует связанные функции
- **Избегает конфликтов имен** - можно иметь две функции с одинаковым именем в разных namespace
- **Улучшает читаемость** - видно, к какой группе относится функция

### 4. Основные команды:

```
// Создание namespace
namespace MyNamespace {
 void myFunction();
}

// Использование (3 способа):
MyNamespace::myFunction(); // Полное имя
using MyNamespace::myFunction; // Для конкретной функции
using namespace MyNamespace; // Для всех функций в namespace
```

### 5. Правила компиляции:

```
Всегда компилируйте ВСЕ .cpp файлы
g++ main.cpp файл1.cpp файл2.cpp -o программа

Если забыли файл, будет ошибка "undefined reference"
```